
CausalFlow: Generating Counterfactual Training Data for LLM Agent Post-Training

Anonymous Authors¹

Abstract

High-quality training data for LLM agents remains scarce because it requires multi-step reasoning traces with tool use and environment interaction, while most synthetic pipelines produce only successful trajectories or uninformative failures. We introduce CausalFlow, a data generation framework that transforms agent failures into contrastive supervision by pairing each failed execution with a minimally edited counterfactual repair. CausalFlow formalizes an agent trace as a dependency DAG and performs step-level interventional attribution to identify failure-inducing steps via Causal Responsibility Scores (CRS): a step is responsible if intervening on it and re-executing downstream steps flips the outcome to the correct answer. Conditioned on CRS-positive steps, we generate multiple repair candidates and select those that are both validated (they succeed under re-execution or outcome verification) and minimal (high token-overlap edits). Across four benchmarks spanning math (GSM8K), code (MBPP), web QA (SealQA Hard), and medical browsing (MedBrowseComp), CausalFlow converts 42.7% of failed runs (555/1299) into validated (failed_trace, repaired_trace) pairs, achieving minimality scores of 0.79–0.87 (see Table 2). The generated dataset includes step-level causal annotations and skill taxonomy labels that support process supervision and curriculum learning, providing a scalable path for agent post-training from real failure traces.

¹Anonymous Institution, Anonymous City, Anonymous Region, Anonymous Country. Correspondence to: Anonymous Author <anon.email@domain.com>.

Preliminary work. Under review by the International Conference on Machine Learning (ICML). Do not distribute.

1. Introduction

LLM-powered agents have emerged as a promising paradigm for autonomous problem-solving, combining language model reasoning with tool use, memory access, and environmental interaction (Yao et al., 2023; Schick et al., 2023; Nakano et al., 2021). However, training agents to perform complex multi-step tasks remains challenging due to a fundamental data scarcity problem: high-quality training data for agentic capabilities is expensive to collect and difficult to generate synthetically.

Current approaches to agent training face three key limitations. First, agentic training data requires multi-step reasoning traces with tool use, which are expensive to manually annotate at scale. Second, typical synthetic generation methods lack error correction signals as they produce either successful traces or failed traces, but not paired examples showing what went wrong and how to fix it. Third, existing data lacks step-level supervision; annotations indicate only whether the final outcome was correct, not which intermediate steps caused failures.

We propose a novel perspective: agent failures are a rich source of training data, not just evaluation artifacts. Every failed execution, when paired with a targeted repair, becomes a contrastive training example. The key is identifying which steps to repair and generating minimal corrections that teach precise fixes rather than wholesale rewrites.

We introduce CausalFlow, a pipeline for generating high-quality contrastive training data from agent failures. Given a failed execution, CausalFlow:

1. Constructs a causal graph from explicit step dependencies
2. Computes Causal Responsibility Scores (CRS) to identify which steps caused the failure
3. Generates minimal counterfactual repairs that fix only the causal steps

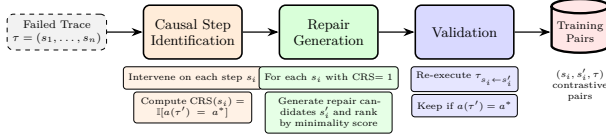


Figure 1. Overview of the CausalFlow pipeline.

Our key insight is that counterfactual repairs provide contrastive supervision: models see both the incorrect reasoning and the corrected version for the same problem. This paired structure, combined with step-level causal annotations, enables more targeted learning than training on successful traces alone. Figure 1 illustrates the complete pipeline.

We evaluate CausalFlow on four diverse benchmarks spanning mathematical reasoning (GSM8K), code generation (MBPP), question answering (SealQA Hard), and medical web browsing (MedBrowseComp), totaling over 3,000 problems. Our pipeline converts 42.7% of failures into validated training pairs with high minimality scores (0.79–0.87). We also organize generated data by a skill taxonomy of 12–16 categories per domain, enabling curriculum-based training.

Contributions. Our main contributions are:

- A pipeline for automatically generating contrastive (failed trace, repaired step) training pairs from agent execution failures (and repaired traces when deterministic re-execution is available)
- Step-level causal annotations via Causal Responsibility Scores (CRS) that identify exactly which steps to learn from
- Quality signals for generated data through minimality constraints on causal attributions
- Extensive experiments across over 3,000 problems converting 42.7% of failures into validated training pairs, with analysis of data quality metrics and skill taxonomies

While our motivation is agent post-training, this work focuses on data generation and quality analysis; evaluating downstream training improvements from CausalFlow-generated data is left for future work.

2. Related Work

Synthetic Data for LLM Training. Generating synthetic training data has become essential for LLM development. Self-Instruct (Wang et al., 2023) generates instruction-following data from seed examples.

RLHF pipelines (Ouyang et al., 2022) create preference data through human feedback. Rejection sampling approaches (Yuan et al., 2023) filter model outputs for quality. Constitutional AI (Bai et al., 2022) uses self-critique to generate preference pairs. However, these methods focus on single-turn or simple multi-turn settings, not complex agentic traces with tool use. CausalFlow addresses this gap by generating contrastive training pairs specifically from agent execution failures.

Process Supervision. Recent work has shown that step-level supervision improves reasoning capabilities. Process reward models (Lightman et al., 2024) train verifiers on human-annotated step correctness. Outcome vs. process supervision (Uesato et al., 2022) demonstrates benefits of intermediate feedback. However, obtaining step-level labels is expensive. CausalFlow automatically generates step-level annotations through causal attribution, identifying which steps caused failures without human labeling.

Causal Inference in Machine Learning. Causal inference provides tools for understanding interventional effects and counterfactual reasoning (Pearl, 2009; Peters et al., 2017). Applications to ML interpretability include causal feature attribution (Schwab & Karlen, 2019), causal concept explanations (Goyal et al., 2019), and causal tracing in transformers (Meng et al., 2022). Our work extends these ideas to the structured domain of agent execution traces, where explicit step dependencies enable direct causal graph construction. We use causal attribution not for explanation but for data generation: identifying which steps to repair to create training pairs.

Error Analysis and Self-Correction. Prior work has analyzed LLM errors through behavioral testing (Ribeiro et al., 2020) or chain-of-thought analysis (Wei et al., 2022). Self-debugging approaches (Chen et al., 2024) have models fix their own errors but lack principled identification of which steps to fix. Our CRS-based approach provides formal grounding for identifying causal steps, enabling targeted repair generation rather than wholesale trace rewriting.

3. Preliminaries

3.1. Agent Execution Traces

We model an LLM agent execution as a sequence of typed steps $\tau = (s_1, s_2, \dots, s_n)$, where each step s_i has:

- A type $t_i \in \mathcal{T}$, where $\mathcal{T} = \{\text{reasoning, tool_call, tool_response, llm_response, memory_access, environment_action, environment_observation, final_answer}\}$
- Type-specific content (e.g., reasoning text, tool name and arguments, or observation data)
- A set of dependencies $D_i \subseteq \{1, \dots, i-1\}$ indicating which prior steps s_i directly depends on

The final step is always of type `final_answer` and represents the agent’s output. We denote the agent’s final answer as $a(\tau)$ and the ground truth as a^* . An execution is successful if $a(\tau) = a^*$ and failed otherwise.

3.2. Causal Graphs from Traces

The dependencies D_i naturally induce a directed acyclic graph (DAG) $G = (V, E)$ where $V = \{s_1, \dots, s_n\}$ and $(s_j, s_i) \in E$ iff $j \in D_i$. This causal graph represents the causal structure of the execution: s_i causally depends on its ancestors in G . In many agent traces this structure is close to linear, but explicit dependencies still provide a principled way to scope local context and downstream effects during interventions, and they generalize to agents with branching tool use or explicit state.

Definition 3.1 (Causal Ancestors). For step s_i , the causal ancestors $\text{Anc}(s_i)$ are all steps reachable by following edges backwards from s_i . These are the steps that could potentially influence s_i .

Definition 3.2 (Causal Steps). The set of causal steps for a failed trace τ is $\mathcal{C}(\tau) = \{s_i : \text{CRS}(s_i) = 1\}$, containing all steps where intervention leads to successful outcome. These are the candidate steps for repair generation.

Definition 3.3 (Contrastive Training Pair). A contrastive training pair (s_i, s'_i, τ) consists of an original failed step s_i , its validated repair s'_i , and the failed trace context τ . When deterministic re-execution is available, we additionally include the repaired trace $\tau_{s_i \leftarrow s'_i}$.

4. The CausalFlow Pipeline

We now present the CausalFlow pipeline for generating training data from agent failures. The pipeline has four components: capturing agent behavior, identifying teachable moments through causal attribution, generating contrastive training pairs, and validating data quality.

4.1. Capturing Agent Behavior

The first stage captures detailed execution traces from agent runs. These traces form the raw material from which training data is generated. Failed traces (where $a(\tau) \neq a^*$) are candidates for training pair generation, as each failure represents an opportunity to create a contrastive example.

4.2. Identifying Teachable Moments

Not every step in a failed trace is worth learning from. Our goal is to identify which steps are causally responsible for the failure these are the “teachable moments” where targeted corrections provide the most training signal. We adopt an interventionist approach inspired by actual causation (Halpern & Pearl, 2005): a step is causally responsible if intervening on it would change the outcome.

Definition 4.1 (Causal Responsibility Score). For a failed trace τ with outcome $a(\tau) \neq a^*$, the Causal Responsibility Score of step s_i is:

$$\text{CRS}(s_i) = \mathbb{I}[a(\tau_{s_i \leftarrow s'_i}) = a^*] \quad (1)$$

where $\tau_{s_i \leftarrow s'_i}$ denotes the trace obtained by replacing s_i with an intervened version s'_i and re-executing all subsequent steps, and $\mathbb{I}[\cdot]$ is the indicator function.

Intuitively, $\text{CRS}(s_i) = 1$ if fixing step s_i (while propagating effects forward) leads to a successful outcome. This provides a binary measure of causal responsibility.

Computing Interventions. To compute s'_i , we prompt an LLM with the original step s_i , a small context window over its explicit dependencies (e.g., the most recent dependent steps), and feedback about the execution failure. The LLM generates a “corrected” version of the step. For reasoning steps, this corrects logical errors; for tool calls, this fixes incorrect arguments or tool selection.

Re-execution. After intervention on s_i , we re-execute the trace from step $i+1$ onwards. This can be done either:

- **Deterministically:** If a domain-specific executor is available (e.g., Python interpreter for code), we re-run the actual computation
- **Predictively:** We use an LLM to predict whether the intervened trace would succeed, based on the corrected step and original context

Algorithm 1 summarizes the CRS computation.

Algorithm 1 Causal Responsibility Scoring

Input: Failed trace $\tau = (s_1, \dots, s_n)$, gold answer a^*
Output: CRS scores $\{\text{CRS}(s_i)\}_{i=1}^{n-1}$
for $i = 1$ to $n - 1$ do
 $s'_i \leftarrow \text{GenerateIntervention}(s_i, \text{DepContext}(s_i), \text{feedback})$
if $s'_i = \emptyset$ then
 $\text{CRS}(s_i) \leftarrow 0$ {No intervention generated}
else
 $\tau' \leftarrow \text{ReExecuteOrPredict}(\tau, i, s'_i)$
 $\text{CRS}(s_i) \leftarrow \mathbb{I}[a(\tau') = a^*]$
end if
end for
return $\{\text{CRS}(s_i)\}$

4.3. Generating Contrastive Training Pairs

For steps with $\text{CRS}(s_i) = 1$ (i.e., causally responsible steps), we generate counterfactual repairs i.e. minimal step-level modifications that, combined with the original failed trace, form contrastive training pairs. In domains where deterministic re-execution is available, we can additionally re-run the agent from the repaired step to obtain a repaired trace; otherwise we retain the repaired step and validate via outcome prediction.

Minimality for Training Quality. A key desideratum is that repairs should be minimal: they should fix only the identified error without introducing unnecessary changes. Minimal repairs ensure models learn targeted corrections rather than wholesale rewrites, making the training signal more precise. We define minimality as:

$$\text{Minimality}(s_i, s'_i) = \frac{m}{L} \left(1 - \frac{1}{2} \cdot \frac{||x| - |y||}{L} \right), \quad (2)$$

where $x = \text{tokens}(s_i)$ and $y = \text{tokens}(s'_i)$ are token sequences, $L = \max(|x|, |y|)$, and $m = \sum_{k=1}^{\min(|x|, |y|)} \mathbb{I}[x_k = y_k]$ counts position-wise token matches. Higher scores indicate smaller edits.

Repair Generation. We generate multiple repair proposals per causal step using temperature sampling, then rank by minimality. The repair prompt emphasizes:

1. Fixing the logical error in the specific step
2. Not using the gold answer directly
3. Making the smallest possible change

Each repair is validated by re-execution (deterministic) or outcome prediction (LLM-based) to confirm it leads to success. Successfully validated pairs are added to the training dataset.

4.4. Training Data Properties

The resulting training data has several properties valuable for agent post-training:

Contrastive Pairs. Each training example contains the failed trace and a minimally repaired version of a causally responsible step. When deterministic re-execution is available, we also include the repaired trace obtained by re-running the agent forward from the repaired step; otherwise the repaired trace is not materialized and we use outcome prediction for validation. This contrastive structure enables models to learn from explicit comparisons between incorrect and corrected behavior.

Step-Level Annotations. CRS provides binary labels indicating which steps caused the failure. This enables process supervision so models can receive feedback at individual steps, not just final outcomes.

Minimal Edits. By ranking repairs by minimality, training pairs teach targeted corrections. Models learn precise fixes rather than learning to completely rewrite traces.

Skill Taxonomy. We cluster labeled causal steps by their error patterns to organize analysis (and optionally training data selection) into skill categories (e.g., arithmetic errors, loop logic bugs, query formulation issues). In our codebase this is implemented as a separate post-processing step over stored traces and analyses.

5. Experiments

We evaluate CausalFlow on four diverse benchmarks to answer:

- Q1 How much training data can CausalFlow generate from agent failures?
- Q2 What is the quality of generated training pairs (minimality, repair rate)?
- Q3 What skill categories does the generated data cover?

5.1. Experimental Setup

Benchmarks. We evaluate on four benchmarks spanning different domains (see Table 1 for full details):

- **GSM8K** (Cobbe et al., 2021): Grade school math word problems requiring multi-step arithmetic reasoning

Table 1. Benchmark setup details. All experiments use the same CausalFlow pipeline with domain-specific validation methods.

Benchmark	Model	Validation
GSM8K	Gemini 2.0	LLM predict
MBPP	GPT-5 Chat	Docker exec
SealQA	Gemini 3 Flash	Re-execution
MedBrowseComp	Gemini 3 Flash	Re-execution

- MBPP (Austin et al., 2021): Python programming problems from the Mostly Basic Programming Problems benchmark
- SealQA Hard (Pham et al., 2025): Complex question answering requiring web search and synthesis
- MedBrowseComp (Chen et al., 2025): Medical question answering with web browsing capabilities

Agent Setup. For each benchmark, we run an appropriate LLM agent:

- GSM8K: Gemini 2.0 Flash Lite with calculator tool
- MBPP: GPT-5 Chat with Docker-based Python execution
- SealQA Hard: Gemini 3 Flash with web search
- MedBrowseComp: Gemini 3 Flash with web browsing

Metrics. We report:

- Repair Success Rate: Fraction of failed executions converted into validated training pairs (this directly measures the pipeline’s utility for training data generation)
- Minimality Score: Average minimality between original and repaired steps (higher = smaller edits)
- Skill Groups: Number of distinct skill categories in generated data

5.2. Training Data Generation Results

Table 2 presents training data generation results across all benchmarks.

CausalFlow achieves its highest repair success rate on GSM8K (52.4%), where errors tend to be localized

Table 2. Training data generation results. CausalFlow converts 42.7% of failures into validated training pairs across four benchmarks. Failed = number of failed agent executions; Pairs = validated (failed, repaired) training pairs generated; Rate = repair success rate (Pairs/Failed); Min. = average minimality score.

Benchmark	Failed	Pairs	Rate	Min.
GSM8K	330	173	52.4%	0.87
MBPP	488	201	41.2%	0.82
SealQA Hard	146	32	21.9%	0.79
MedBrowseComp	335	149	44.5%	0.84
Total	1299	555	42.7%	0.83

arithmetic or reasoning mistakes amenable to minimal repair. GSM8K also achieves the highest minimality score (0.87). The lowest rate is on SealQA Hard (21.9%), where failures often involve information retrieval gaps that cannot be fixed by modifying reasoning steps alone.

Training Pair Characteristics. Generated training pairs exhibit several patterns:

- Arithmetic corrections (GSM8K): Pairs showing incorrect $\rightarrow$ correct calculation steps
- Logic bug fixes (MBPP): Pairs demonstrating off-by-one error corrections, boundary condition handling
- Query refinement (SealQA, MedBrowseComp): Pairs showing improved search query formulation
- Reasoning corrections: Pairs fixing incorrect inferences or missed constraints

5.3. Skill Taxonomy for Curriculum Learning

CausalFlow’s causal attributions reveal skill categories that organize generated training data. We cluster causal steps by their error patterns to create a skill taxonomy, enabling curriculum-based training (Table 3). GSM8K has the most diverse categories (16), including arithmetic operations, unit conversions, multi-step planning, and constraint tracking. MBPP clusters into 12 categories covering loop logic, string manipulation, and edge case handling. This taxonomy enables targeted training: practitioners can select training pairs for specific skills their models struggle with, or construct curricula progressing from simpler to more complex skill categories.

5.4. Qualitative Examples

Example 1: Off-by-One Error in Break-Even Analysis. Consider this problem from GSM8K: “Carlos is plant-

ing a lemon tree. The tree will cost \$90 to plant. Each year it will grow 7 lemons, which he can sell for \$1.5 each. It costs \$3 a year to water and feed the tree. How many years will it take before he starts earning money on the lemon tree?"

The agent trace shows:

1. Reasoning: "Calculate annual income: 7 lemons $\times$ \$1.5 = \$10.50"
2. Tool call: calculator($7 * 1.5$) = 10.5
3. Reasoning: "Calculate net profit per year: \$10.50 - \$3 = \$7.50"
4. Tool call: calculator($10.5 - 3$) = 7.5
5. Reasoning: "Years to cover initial cost: \$90 $\div$ \$7.50 = 12"
6. Tool call: calculator($90 / 7.5$) = 12
7. Final answer: 12 [INCORRECT: gold answer is 13]

The error is subtle: 12 years of \$7.50 profit exactly covers the \$90 cost (break-even), but the question asks when Carlos starts earning money i.e., when cumulative profit first becomes positive, not zero. At year 13, he has his first positive earnings.

CausalFlow identifies Step 5 as causal and generates the validated repair: "...Divide the initial cost by the annual net profit to find the number of years. Since we are looking for when he starts earning money i.e., when cumulative profit first becomes positive, if the division results in a whole number, we need to add one year."

Training Value. This contrastive pair teaches models to distinguish break-even from positive-profit conditions, a common off-by-one reasoning error. The repair targets the planning step where the boundary condition was missed rather than simply changing the final answer.

Example 2: MBPP Logic Bug. A function to check if a number is prime incorrectly returns True for 1:

```
def is_prime(n):
    for i in range(2, n):
        if n % i == 0:
            return False
    return True # Bug: n=1 returns True
```

CausalFlow traces the failure to the missing base case and generates a minimal repair adding if $n \leq 1$: return False.

Table 3. Skill taxonomy for generated training data. Numbers indicate distinct categories for organizing training pairs by skill deficit. Categories are derived from clustering causal steps by error patterns.

Benchmark	Skills	Example Categories
GSM8K	16	arithmetic, unit conv.
MBPP	12	loop logic, strings
SealQA Hard	4	query, synthesis
MedBrowseComp	6	med. reasoning

Training Value. This pair teaches edge case handling: the original trace shows the common pattern of forgetting boundary conditions, and the repair demonstrates the minimal fix. The contrastive structure helps models learn to check base cases when implementing mathematical predicates.

5.5. Ablation Studies

Minimality Constraint. Without minimality ranking, repairs tend to involve larger rewrites. While these repairs still lead to correct answers, they provide a weaker training signal because models learn to rewrite traces rather than make targeted fixes. The minimality constraint ensures training pairs teach precise corrections.

6. Discussion

Key Findings. Our experiments reveal several insights about generating training data from agent failures:

1. Failures are often localized: In most cases, 1–3 steps are causally responsible, enabling targeted training pairs rather than wholesale trace rewrites
2. Generation rates vary by domain: Closed-domain tasks (GSM8K, MBPP) yield higher-quality training data than open-domain retrieval tasks
3. Skill taxonomies emerge: Causal analysis naturally organizes training data into interpretable skill categories for curriculum learning

Implications for Agent Training. The contrastive training pairs generated by CausalFlow enable several training approaches:

- Process supervision: Step-level CRS annotations provide supervision signal at intermediate steps, not just final outcomes
- Curriculum learning: Skill taxonomies enable training curricula that target specific capability

gaps

- Contrastive learning: (Failed, repaired) pairs can train models to distinguish correct from incorrect reasoning
- Self-improvement: Models can be trained on repairs of their own failures, creating a closed-loop improvement cycle

Limitations. CausalFlow has several limitations:

- Repair quality: LLM-generated repairs may not always represent optimal fixes; we validate repairs via deterministic re-execution when available or LLM-based outcome prediction otherwise
- Generation costs: Computing CRS requires re-running (parts of) traces, which can be expensive for complex agents
- Retrieval failures: When failures stem from information not being retrievable (vs. reasoning errors), useful training pairs cannot be generated
- Coverage: Not all agent failures can be converted to training pairs; some require architectural changes rather than step repairs

Future Work. Promising directions include: (1) training agents on CausalFlow-generated data to measure downstream improvement, (2) active learning to prioritize failure cases most valuable for training, (3) extending to multi-agent systems where failures involve coordination, and (4) learning to predict CRS without re-execution to reduce generation costs.

7. Conclusion

We presented CausalFlow, a pipeline for generating high-quality contrastive training data from LLM agent failures. By applying causal inference to identify which steps caused failures and generating minimal counterfactual repairs, CausalFlow transforms agent failures into (failed trace, repaired step) training pairs (and repaired traces when deterministic re-execution is available). Experiments on four benchmarks (over 3,000 total problems) demonstrate that CausalFlow converts 42.7% of failures into validated training pairs with high minimality scores (0.79–0.87), as shown in Table 2. The repair success rate directly measures the pipeline’s utility: each validated pair represents a failure successfully converted into contrastive supervision. The dataset includes step-level causal annotations, and we additionally derive skill taxonomies for curriculum

learning via post-processing (Table 3). We believe automated generation of contrastive training data from failures is a promising direction for scalable agent post-training.

Impact Statement

This paper presents work whose goal is to advance the field of Machine Learning, specifically in generating training data for improving LLM agent capabilities. CausalFlow enables practitioners to automatically generate high-quality training data from agent failures, potentially reducing the cost and effort required to improve agent systems.

The generated training data could be used to make agents more capable, which has both positive applications (more reliable AI assistants) and potential risks (more capable systems may be misused). We believe the benefits of improved agent training outweigh these risks, as the same training improvements could be achieved through more expensive manual annotation. Our work reduces costs but does not enable fundamentally new capabilities.

References

- Austin, J., Odena, A., Nye, M., Bosma, M., Michalewski, H., Dohan, D., Jiang, E., Cai, C., Terry, M., Le, Q., et al. Program synthesis with large language models. arXiv preprint arXiv:2108.07732, 2021.
- Bai, Y., Kadavath, S., Kundu, S., Askell, A., Kernion, J., Jones, A., Chen, A., Goldie, A., Mirhoseini, A., McKinnon, C., et al. Constitutional AI: Harmlessness from AI feedback. arXiv preprint arXiv:2212.08073, 2022.
- Chen, S., Moreira, P., Xiao, Y., Schmidgall, S., Warner, J., Aerts, H., Hartvigsen, T., Gallifant, J., and Bitterman, D. S. MedBrowseComp: Benchmarking medical deep research and computer use. arXiv preprint arXiv:2505.14963, 2025.
- Chen, X., Lin, M., Schärli, N., and Zhou, D. Teaching large language models to self-debug. International Conference on Learning Representations, 2024.
- Cobbe, K., Kosaraju, V., Bavarian, M., Chen, M., Jun, H., Kaiser, L., Plappert, M., Tworek, J., Hilton, J., Nakano, R., et al. Training verifiers to solve math word problems. arXiv preprint arXiv:2110.14168, 2021.
- Goyal, Y., Feder, A., Shalit, U., and Kim, B. Explain-

- ing classifiers with causal concept effect (CaCE). In arXiv preprint arXiv:1907.07165, 2019.
- Halpern, J. Y. and Pearl, J. Causes and explanations: A structural-model approach. part I: Causes. *The British Journal for the Philosophy of Science*, 56(4): 843–887, 2005.
- Lightman, H., Kosaraju, V., Burda, Y., Edwards, H., Baker, B., Lee, T., Leike, J., Schulman, J., Sutskever, I., and Cobbe, K. Let’s verify step by step. *International Conference on Learning Representations*, 2024.
- Meng, K., Bau, D., Andonian, A., and Belinkov, Y. Locating and editing factual associations in GPT. In *Advances in Neural Information Processing Systems*, 2022.
- Nakano, R., Hilton, J., Balaji, S., Wu, J., Ouyang, L., Kim, C., Hesse, C., Jain, S., Kosaraju, V., Saunders, W., et al. WebGPT: Browser-assisted question-answering with human feedback. arXiv preprint arXiv:2112.09332, 2021.
- Ouyang, L., Wu, J., Jiang, X., Almeida, D., Wainwright, C., Mishkin, P., Zhang, C., Agarwal, S., Slama, K., Ray, A., et al. Training language models to follow instructions with human feedback. *Advances in Neural Information Processing Systems*, 35:27730–27744, 2022.
- Pearl, J. *Causality: Models, Reasoning, and Inference*. Cambridge University Press, 2nd edition, 2009.
- Peters, J., Janzing, D., and Schölkopf, B. *Elements of Causal Inference: Foundations and Learning Algorithms*. MIT Press, 2017.
- Pham, T., Nguyen, N., Zunjare, P., Chen, W., Tseng, Y.-M., and Vu, T. SealQA: Raising the bar for reasoning in search-augmented language models. arXiv preprint arXiv:2506.01062, 2025.
- Ribeiro, M. T., Wu, T., Guestrin, C., and Singh, S. Beyond accuracy: Behavioral testing of NLP models with CheckList. In *Association for Computational Linguistics*, 2020.
- Schick, T., Dwivedi-Yu, J., Dessì, R., Raileanu, R., Lomeli, M., Hambro, E., Zettlemoyer, L., Cancedda, N., and Scialom, T. Toolformer: Language models can teach themselves to use tools. In *Advances in Neural Information Processing Systems*, volume 36, 2023.
- Schwab, P. and Karlen, W. CXPlain: Causal explanations for model interpretation under uncertainty. In *Advances in Neural Information Processing Systems*, 2019.
- Uesato, J., Kushman, N., Kumar, R., Song, F., Siegel, N., Wang, L., Creswell, A., Irving, G., and Higgins, I. Solving math word problems with process-and outcome-based feedback. arXiv preprint arXiv:2211.14275, 2022.
- Wang, Y., Kordi, Y., Mishra, S., Liu, A., Smith, N. A., Khashabi, D., and Hajishirzi, H. Self-instruct: Aligning language models with self-generated instructions. *Association for Computational Linguistics*, 2023.
- Wei, J., Wang, X., Schuurmans, D., Bosma, M., Ichter, B., Xia, F., Chi, E., Le, Q., and Zhou, D. Chain-of-thought prompting elicits reasoning in large language models. *Advances in Neural Information Processing Systems*, 2022.
- Yao, S., Zhao, J., Yu, D., Du, N., Shafran, I., Narasimhan, K., and Cao, Y. ReAct: Synergizing reasoning and acting in language models. *International Conference on Learning Representations*, 2023.
- Yuan, Z., Yuan, H., Li, C., Dong, G., Lu, K., Tan, C., Zhou, C., and Zhou, J. Scaling relationship on learning mathematical reasoning with large language models. arXiv preprint arXiv:2308.01825, 2023.

A. Experimental Details

This section provides complete experimental setup details for each benchmark, including dataset specifications, agent configurations, prompts used, and validation methods.

A.1. GSM8K

Dataset. We use the GSM8K dataset (Cobbe et al., 2021) loaded from HuggingFace (gsm8k, main config, test split), comprising 1,319 grade-school math word problems.

Model.

- model_used: google/gemini-2.0-flash-lite-001
- Temperature: 0.7

Agent Configuration. The GSM8K agent (GSM8KAgent) decomposes problems into structured steps:

1. Initial reasoning step stating the problem
2. LLM generates structured solution with reasoning, steps (each with description, operation, expression), and final answer
3. For each calculation step: reasoning $\rightarrow$ calculator tool call $\rightarrow$ tool response
4. Final answer step

Tools.

- calculator: Evaluates mathematical expressions using MathReexecutor

Prompts. The solve prompt requests:

Solve this math problem step by step.

Problem: {question}

Provide:

1. A brief reasoning about your approach
2. Each calculation step with:
 - A clear description
 - The operation type (addition, subtraction, multiplication, division, or other)
 - The exact mathematical expression to evaluate
3. The final numerical answer

Be precise with expressions - they should be evaluable.

Validation Method. LLM-based outcome prediction (no deterministic re-executor). Gold answers are extracted as numeric values using MathReexecutor.extract_number().

CausalFlow Configuration.

- Intervene on step types: TOOL_CALL, LLM_RESPONSE, REASONING, TOOL_RESPONSE

A.2. MBPP

Dataset. We use the MBPP dataset (Austin et al., 2021) loaded from HuggingFace (Muennighoff/mbpp), with train/test/validation/prompt splits merged. Total problems: 974 (after filtering).

Model.

- model_used: GPT 5 Chat
- Temperature: 0.2 for code generation

Agent Configuration. The MBPP agent reuses the HumanEval-style code generator (HumanevalAgent):

1. Reasoning step: “Implement {function} for task {id}”
2. Tool call: llm_code_generation
3. LLM response: generated Python code
4. Tool call: docker_code_execution
5. Tool response: test results
6. Final answer: “pass” or “fail”

Tools.

- llm_code_generation: Generates Python code via LLM
- docker_code_execution: Runs code in isolated Docker container with official test cases

Prompts. Code generation prompt:

Complete the following Python function. Return only executable Python code.
{prompt}

Rules:

- Function name must be exactly ‘{entry_point}’
- Preserve provided signature and docstring
- Include required imports
- No extra prints or explanations

Validation Method. Deterministic Docker-based execution using HumanevalReexecutor. Tests are run in isolated containers; success requires all test assertions to pass.

CausalFlow Configuration.

- Deterministic re-executor: HumanevalReexecutor with Docker

A.3. SealQA Hard

Dataset. We use SealQA Hard (Pham et al., 2025) loaded from HuggingFace (vllms/sealqa, seal_hard config, test split), comprising 254 complex QA problems requiring web search.

Model.

- model_used: google/gemini-3-flash-preview
- Temperature: 0.0 for solver, 0.3 for agent steps

Agent Configuration. The BrowseComp agent (BrowseCompAgent) uses a structured tool policy:

- Maximum steps: 15
- Actions: search, open_url, extract, answer
- State tracking: gathered facts, search history, page summaries

Tools.

- web_search: Search via Serper API with caching
- web_fetch: Fetch and parse web pages

Prompts. Agent system prompt:

You are a web browsing agent tasked with answering questions by searching the web and examining pages.

You have access to the following tools:

1. search(query) - Search the web for information
2. open_url(url) - Fetch and read the content of a specific URL
3. extract - Note important facts from the current context
4. answer - Provide your final answer when you have enough information

Important guidelines:

- Start by searching for relevant information
- Open URLs that seem most likely to contain the answer
- Extract key facts as you find them
- When you have enough information, provide your answer

Grader template:

You are an answer checker. Use ONLY the provided [response] and [correct_answer].

Response: {response}

Correct Answer: {correct_answer}

extracted_final_answer: Extract the final answer from Response.

reasoning: Compare extracted_final_answer against Correct Answer.

correct: 'yes' if equivalent; otherwise 'no'.

Validation Method. LLM-based grading using the grader template. Web results are cached in .cache/browsecomp for reproducibility.

CausalFlow Configuration.

- Intervene on step types: TOOL_CALL, LLM_RESPONSE, REASONING
- Web logs truncated to 15,000 characters before analysis

A.4. MedBrowseComp

Dataset. We use MedBrowseComp (Chen et al., 2025) loaded from HuggingFace (AIM-Harvard/MedBrowseComp_CUA, MedBrowseComp_CUA split), comprising 484 medical QA problems. Problems are sampled with random.Random(42) when a subset is requested.

Model.

- model_used: google/gemini-3-flash-preview
- Temperature: 0.3 for agent steps

Agent Configuration. Same as SealQA (BrowseCompAgent) with:

- Maximum steps: 10 (reduced for medical domain)
- Same tool policy and state tracking

Tools. Same as SealQA: web_search, web_fetch

Validation Method. LLM-based grading using the same grader template as SealQA.

CausalFlow Configuration.

- Intervene on step types: TOOL_CALL, LLM_RESPONSE, REASONING

B. Implementation Details

B.1. Trace Logger

The TraceLogger captures agent execution through typed step logging:

```
class StepType(Enum):
    REASONING = "reasoning"
    TOOL_CALL = "tool_call"
    TOOL_RESPONSE = "tool_response"
    LLM_RESPONSE = "llm_response"
    MEMORY_ACCESS = "memory_access"
    ENVIRONMENT_ACTION = "environment_action"
    ENVIRONMENT_OBSERVATION = "environment_observation"
    FINAL_ANSWER = "final_answer"
```

Each step records its type, content, and explicit dependencies on prior steps.

B.2. Causal Graph Construction

The causal graph is constructed directly from step dependencies using NetworkX:

```
def _build_graph(self):
    for step in self.trace.steps:
        self.graph.add_node(step.step_id, step_type=step.step_type.value)
    for step in self.trace.steps:
        for dep_id in step.dependencies:
            self.graph.add_edge(dep_id, step.step_id)
```

B.3. Intervention Prompts

The intervention prompt for CRS computation follows this template:

System: You are an expert at debugging agent reasoning steps.
Given a step that contributed to a failed execution, suggest
a corrected version that would lead to success.

Context: [Previous steps in trace]
 Failed Step: [Step content]
 Feedback: [Execution failure message]

Generate a minimal correction to this step.

B.4. Training Pair Validation

Training pairs are validated through either deterministic re-execution (when an executor is available) or LLM-based outcome prediction:

```
def _evaluate_repair_success(self, step_id, repaired_step):
    if self.reexecutor is None:
        # LLM-based outcome prediction (no repaired trace available)
        return self._llm_predict_outcome(step_id, repaired_step)

    # Deterministic validation: re-run from the repaired step forward
    history = steps_before(step_id) + [repaired_step]
    new_trace = self.reexecutor.run_remaining_steps(history)
    return new_trace.success
```

Only pairs where the repair leads to successful execution are included in the final training dataset.

C. Extended Results

C.1. Dataset and Trace Statistics

Table 4 provides detailed statistics for each benchmark run.

Table 4. Dataset and trace statistics. Total / Avg. row shows sums for counts and macro-averages for rates.

Benchmark	Total	Passed	Failed	Fixed	Fix Rate	Accuracy
GSM8K	1319	989	330	173	52.4%	75.0%
MBPP	947	523	488	201	41.2%	55.2%
SealQA Hard	254	108	146	32	21.9%	42.5%
MedBrowseComp	484	149	335	149	44.5%	30.8%
Total / Avg.	3004	1769	1299	555	40.0%	50.8%

C.2. Per-Model Training Data Generation

Table 5. Training data generation broken down by base model used for agent execution.

Benchmark	Model	Failed	Pairs	Rate	Min.
GSM8K	Gemini 2.0 Flash Lite	330	173	52.4%	0.87
MBPP	GPT-5 Chat	488	201	41.2%	0.82
SealQA Hard	Gemini 3 Flash	146	32	21.9%	0.79
MedBrowseComp	Gemini 3 Flash	335	149	44.5%	0.84

C.3. Skill Taxonomy Details

The following skill categories organize generated training data for curriculum learning. Category counts are derived from clustering causal steps using LLM-based skill labeling and grouping.

GSM8K Categories (16): Basic arithmetic, multi-digit multiplication, division with remainders, unit conversion, percentage calculation, ratio and proportion, multi-step planning, constraint satisfaction, word problem

parsing, variable tracking, comparison operations, time calculations, money calculations, distance/rate/time, combinatorics basics, and equation setup.

MBPP Categories (12): Task decomposition and planning, algorithmic logic implementation, tool and API integration, code generation and syntactic integrity, requirement interpretation and mapping, data structure and type management, edge case and boundary handling, mathematical and geometric reasoning, regex and pattern synthesis, input parsing and verification, computational efficiency optimization, and unassigned/outlier failures.

SealQA Hard Categories (4): Query precision and formulation, source selection and extraction efficiency, iterative search and refinement, and temporal/contextual grounding.

MedBrowseComp Categories (6): Source relevance and authority assessment, iterative query refinement and narrowing, search query optimization and formulation, information extraction and parsing, redundancy and execution efficiency, and identifier/resource verification.

Using the Taxonomy. Practitioners can use these categories to: (1) identify which skills their models struggle with, (2) select training pairs targeting specific deficits, and (3) construct curricula that progress from simpler to more complex skills within each domain.

D. Reproducibility

All experiments were run using the CausalFlow codebase. Key configuration:

- API Access: OpenRouter API for LLM calls; Serper API for web search
- Storage: MongoDB for trace and analysis persistence
- Execution: Docker containers for MBPP/HumanEval code execution

Run scripts are located in:

- experiments/gsm8k/run_gsm8k_experiment.py
- experiments/mbpp/run_mbpp_experiment.py
- experiments/browsecomp/run_sealqa_experiment.py
- experiments/browsecomp/run_medbrowsecomp_experiment.py